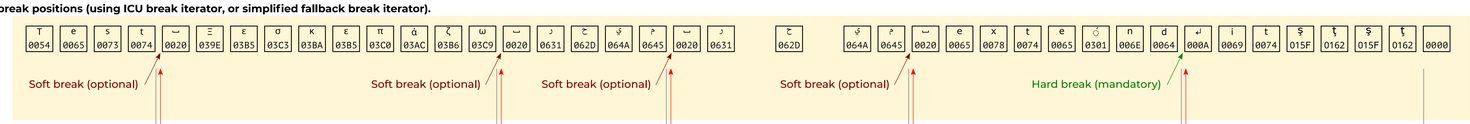
2. Line breaking.

2.1. Locate valid break positions (using ICU break iterator, or simplified fallback break iterator).



2.3.2. Break line at to last fitting valid break.

Last fitting valid break

2.2. Reorder clusters in the logical order. (sort by code unit positions)

2.3.1. Measure clusters until target width is reached.

2.3. Fit to target width.

Width

2.3.3. If mandatory break is encountered break at any line width.

Mandatory break

EOL

ICU Break iterator API
ICU Boundary Analysis
UAX #14: Unicode Line Breaking Algorithm

Some languages don't use spaces between the words.

Line bounds:

Line 1: 0...4
Line 2: 5...14
Line 3: 15...19
Line 4: 20...24
Line 5: 25...31
Line 6: 33...39

2.4. Rerun BiDi on each line.

Rerun BiDi and reshape each line:

Script runs can be reused.
ICU BiDi context (but not BiDi algorithm results!) can be reused as well.

1 HarfBuzz

Reshape line

Fully identical runs can be reused without reshaping.

2.4. Rershape each line.

Output

Test
Ξεσκεπάζω
رحيم
رحيم
exténd
itşţşţ

1

Test Ξεσκεπάζω رحيم رحيم exténd itşţşţ

Run order changed after line breaking!

Break opportunies search code

Generate break opportunies

```
UBreakIterator *bi = ubrk_open(UBRK_LINE, lang, data, data.size, &err);
if (U_FAILURE(err)) {
    //No ICU data - use fallback
    for (int64_t i = 0; i < data.size; i++) {
        if (is_break(get_char(i))) {

            op.position = i + 1;
            op.hard = true;

        } else if (u_isWhitespace(get_char(i))) {

            op.position = i + 1;
            op.hard = false;

        }
    }
}
while (ubrk_next(bi) != UBRK_DONE) {

            op.position = ubrk_current(bi);
            op.hard = (ubrk_getRuleStatus(bi) == UBRK_LINE_HARD);

}
ubrk_close(bi);
```

Sort clusters in logical order

```
std::vector<Cluster> logical = visual;
std::sort(logical.begin(), logical.end(), ClusterCompare);
```

Line breaking

Break lines

```
float width = 0.0f;
int64_t line_start = 0;

int64_t last_safe_brk = -1;
int64_t last_safe_brk_cluster = -1;

int64_t b = 0;
int64_t i = 0;
while (i < logical.size()) {
    if ((b < brk_ops.size()) && (brk_ops[b].position == logical[i].start)) {
        last_safe_brk = b;
        last_safe_brk_cluster = i;
        b++;
        if (brk_ops[last_safe_brk].hard) {

            logical[i].end

            width = 0.0f;
            line_start = logical[i].end;
            last_safe_brk = -1;
            last_safe_brk_cluster = -1;
            i++;
            continue;
        }
    }
    width += logical[i].width;
    if (p_flags == TEXT_BREAK_MANDATORY_AND_WORD_BOUND) {
        if ((p_width > 0) && (width >= p_width) && (last_safe_brk != -1) && (brk_ops[last_safe_brk].position != line_start)) {

            brk_ops[last_safe_brk].position

            width = 0.0f;
            i = last_safe_brk_cluster;
            line_start = brk_ops[last_safe_brk].position;
            last_safe_brk = -1;
            last_safe_brk_cluster = -1;
            continue;
        }
    } else if (p_flags == TEXT_BREAK_MANDATORY_AND_ANYWHERE) {
        if ((p_width > 0) && (width >= p_width)) {

            logical[i].end

            width = 0.0f;
            line_start = logical[i].end;
            last_safe_brk = -1;
            last_safe_brk_cluster = -1;
            i++;
            continue;
        }
    }
    i++;
}
```

Init line BiDi iterator

```
UBiDi *bidi_iter = ubidi_openSized((p_end - p_start), 0, &err);
if (U_FAILURE(err)) {
    bidi_iter = NULL;
    //Do not error out on failure - just continue with full reshapeing
} else {
    ubidi_setline(bidi_iter, p_start, p_end, bidi_iter, &err);
    if (U_FAILURE(err)) {
        ubidi_close(p_ref->bidi_iter);
        bidi_iter = NULL;
    }
}
```

gdtl module source code links

general documentation/info links

CC 0